



**UNIVERSIDAD TECNOLÓGICA
DE QUERÉTARO**

Alumna:

Vasquez Noyola Andrea

Grupo:

IDGS17

Matrícula:

2023148002

Materia:

Administración de Base de Datos

**Práctica de Laboratorio: Planes de Administración,
Respaldo y Automatización en DBMS Relacionales
(PostgreSQL) - Caso 9**

1. Tabla Comparativa del OWASP Top 10 2023	3
2. Casos Reales del Contexto de la Industria (2 por vulnerabilidad)	6
3. Código Vulnerable vs. Seguro (JavaScript)	8
Ejemplo 1: A03 - Inyección SQL (SQL Injection)	8
Código Vulnerable (Concatenación Directa)	8
Código Seguro (Consultas Parametrizadas / Prepared Statements)	9
Ejemplo 2: A01 - Broken Access Control (Insecure Direct Object Reference - IDOR)	10
Código Vulnerable (Falta de Verificación de Propietario)	10
Código Seguro (Validación del Contexto del Usuario Autenticado)	11
Ejemplo 3: A05 - Security Misconfiguration (Exposición Extensa de Errores)	11
Código Vulnerable (Debug Mode y Stack Traces Expuestos en Producción)	11
Código Seguro (Mensajes Genéricos al Cliente y Logging Privado)	12
4. Ranking Personal de las 5 Vulnerabilidades Más Críticas	13
5. Conclusión	14
Bibliografías	15

1. Tabla Comparativa del OWASP Top 10 2023

Código	Nombre de la Vulnerabilidad	Descripción Corta	Ejemplo de Ataque	Impacto Potencial	Método de Mitigación
A01	Broken Access Control (Control de Acceso Completo)	Los usuarios pueden acceder a recursos o funciones fuera de sus permisos previstos debido a una mala verificación de autorización.	Modificar el ID de usuario en la URL (/user/profile?id=1002 a id=1003) para ver perfiles ajenos (IDOR).	Fuga masiva de datos confidenciales, manipulación de cuentas y escalación ilícita de privilegios.	Implementar el principio de menor privilegio, forzar control de acceso basado en el servidor y denegar por defecto.
A02	Cryptographic Failures (Fallas Criptográficas)	Exposición indebida de datos sensibles (passwords, tarjetas, datos de salud) por falta de cifrado o algoritmos obsoletos.	Interceptar tráfico HTTP en texto plano (ataque MitM) o extraer hashes de contraseñas guardados con MD5.	Robo de identidad, fraude financiero, sanciones legales por brechas de datos personales.	Cifrar datos en tránsito (HTTPS) y en reposo (AES), usar funciones de hash modernas como Argon2 o bcrypt.
A03	Injection (Inyección)	Datos no confiables proporcionados por el usuario son interpretados directamente como comandos o consultas por un intérprete.	Introducir ' OR '1'='1 en un formulario de login para evadir la validación de la base de datos SQL.	Ejecución remota de código (RCE), borrado de bases de datos o evasión completa de autenticación.	Usar consultas parametrizadas (Prepared Statements), APIs seguras y validación estricta de entradas (listas blancas).

A04	Insecure Design (Diseño Inseguro)	Defectos inherentes en la arquitectura y el modelado de amenazas del software antes de escribir el código fuente.	Crear un sistema de recuperación de contraseñas basado en preguntas simples y predecibles (ej. "¿Cuál es tu color favorito?").	Vulnerabilidades estructurales masivas que no pueden repararse con parches de código simples.	Implementar metodologías de ciclo de vida seguro (SDLC), modelado de amenazas y patrones de diseño validados.
A05	Security Misconfiguration (Configuración Incorrecta de Seguridad)	Endurecimiento (hardening) deficiente de los servidores, uso de configuraciones por defecto o visualización de errores detallados.	Dejar activadas las credenciales de fábrica (admin/admin) en la consola de administración de un servidor web.	Acceso completo de atacantes a servidores, exposición de stack traces de desarrollo con rutas del sistema.	Deshabilitar funciones innecesarias, cambiar credenciales por defecto y automatizar auditorías de infraestructura.
A06	Vulnerable and Outdated Components (Componentes Vulnerables e Obsoletos)	Utilizar bibliotecas, dependencias o frameworks en el proyecto que contengan vulnerabilidades conocidas públicamente.	Mantener una versión antigua de una dependencia de Node.js (npm) que posee un exploit público para denegación de servicio.	Compromiso total de la aplicación a través de fallas conocidas en software de terceros de la cadena de suministro.	Mantener un inventario continuo (Sbom), usar herramientas de escaneo como npm audit o OWASP Dependency-Check.

A07	Identification and Authentication Failures (Fallas de Identificación y Autenticación)	Debilidades en la gestión de sesiones o validación de la identidad del usuario, permitiendo la suplantación.	Ataques automatizados de fuerza bruta (Credential Stuffing) debido a la ausencia de límites de intentos o CAPTCHAs.	Secuestro de sesiones activas, robo masivo de cuentas de usuario por vectores automatizados.	Implementar Autenticación de Múltiples Factores (MFA), políticas estrictas de contraseñas y rotación segura de tokens de sesión.
A08	Software and Data Integrity Failures (Fallas en la Integridad de Software y Datos)	Código e infraestructura que no protegen contra violaciones de integridad (ej. actualizaciones sin firmar o deserialización insegura).	Modificar un objeto serializado almacenado en una cookie de sesión de JavaScript para cambiar el rol de user a admin.	Ejecución remota de código, inyección de artefactos maliciosos en actualizaciones automáticas.	Utilizar firmas digitales para validar actualizaciones y evitar serializar objetos provenientes de fuentes no confiables.
A09	Security Logging and Monitoring Failures (Fallas en el Registro y Monitoreo de Seguridad)	No registrar eventos críticos de seguridad o registrar bitácoras sin la capacidad de generar alertas ante ataques en tiempo real.	Un atacante intenta miles de logins durante semanas, pero el sistema no genera alarmas ni registra las IPs atacantes.	Incapacidad para detectar brechas activas; los atacantes pueden permanecer meses dentro del sistema sin ser detectados.	Registrar logins fallidos, cambios de privilegios, y centralizar los logs en sistemas SIEM con alertas en tiempo real.

A10	Server-Side Request Forgery (SSRF)	La aplicación web induce al servidor a realizar una petición HTTP hacia un dominio arbitrario o la red interna.	Proveer una URL interna (http://localhost:8080/admin) en una funcionalidad del perfil que descarga imágenes por URL.	Acceso de atacantes a servicios internos protegidos detrás de firewalls, escaneo de la red local del servidor.	Validar y desinfectar URLs provistas por el usuario, aplicar listas blancas de dominios y restringir accesos de red del servidor.
------------	------------------------------------	---	--	--	---

2. Casos Reales del Contexto de la Industria (2 por vulnerabilidad)

A01: Broken Access Control

- *Caso 1 (First American Financial, 2019):* Una falla de IDOR expuso más de 885 millones de documentos financieros adjuntos a URLs. Modificando secuencialmente el número del documento en la barra del navegador, cualquier usuario web anónimo podía descargar historiales fiscales e identificaciones sin autenticación.
- *Caso 2 (Twitter/X, 2021):* Un fallo en una API permitía a un atacante introducir números de teléfono o correos electrónicos para averiguar qué ID de cuenta interna de Twitter correspondía a dichos datos, vulnerando el control de acceso a la privacidad y afectando a millones de usuarios.

A02: Cryptographic Failures

- *Caso 1 (Ashley Madison, 2015):* Al ser vulnerada la base de datos de la plataforma, se descubrió el uso del algoritmo obsoleto e inseguro MD5 para generar hashes de contraseñas de millones de usuarios, facilitando el descifrado rápido de las credenciales expuestas mediante tablas de búsqueda.
- *Caso 2 (Capital One, 2019):* Una mala configuración criptográfica de almacenamiento de datos combinada con un ataque SSRF expuso los números de seguro social y cuentas bancarias de más de 100 millones de clientes al no contar con un cifrado de datos robusto en reposo dentro de los buckets de AWS.

A03: Injection

- *Caso 1 (Sony Pictures, 2011):* El grupo LulzSec extrajo datos confidenciales de más de un millón de cuentas usando una inyección SQL simple en formularios web no desinfectados de la página corporativa, comprometiendo bases de datos enteras de cupones y passwords.

- *Caso 2 (Equifax, 2017)*: Aunque comúnmente asociado a componentes obsoletos, la causa raíz del vector inicial de entrada fue una Inyección de Comandos de Apache Struts (CVE-2017-5638). Un atacante introdujo comandos maliciosos en la cabecera HTTP Content-Type que el servidor interpretó directamente, robando datos de 147 millones de personas.

A04: Insecure Design

- *Caso 1 (Zoom, 2020)*: El fenómeno conocido como "Zoombombing" se originó debido a que el diseño inicial de las salas de conferencias no requería contraseñas de acceso ni salas de espera por defecto, permitiendo que atacantes adivinaran IDs de reuniones numéricas para sabotearlas con contenido ofensivo.
- *Caso 2 (SolarWinds, 2020)*: Fallas estructurales en el diseño de su pipeline de CI/CD permitieron al malware "SUNBURST" inyectarse en el código fuente de actualización legítimo sin levantar alarmas del sistema de empaquetado, reflejando una falta de diseño enfocado en la resiliencia de la cadena de suministro.

A05: Security Misconfiguration

- *Caso 1 (Dow Jones, 2017)*: Un bucket de almacenamiento de Amazon S3 configurado incorrectamente con permisos de visualización para "Cualquier usuario autenticado de AWS" expuso datos personales y financieros de más de 2.2 millones de suscriptores del Wall Street Journal.
- *Caso 2 (Elasticsearch Breaches, 2018-2020)*: Múltiples empresas sufrieron fugas masivas debido a que las instancias de Elasticsearch venían de fábrica sin autenticación requerida en el puerto 9200. Servidores expuestos a Internet pública permitieron el secuestro y borrado de bases de datos por bots automatizados.

A06: Vulnerable and Outdated Components

- *Caso 1 (Equifax, 2017)*: La compañía omitió parchar de inmediato la vulnerabilidad crítica en el framework Apache Struts, a pesar de que el parche de seguridad público ya tenía meses disponible, derivando en pérdidas financieras multimillonarias y exposición de historiales crediticios.
- *Caso 2 (Log4Shell - Log4j, 2021)*: Prácticamente miles de empresas de tecnología (incluidas iCloud, Minecraft, y servidores empresariales) descubrieron que utilizaban la librería de registros Java log4j obsoleta y vulnerable a ejecución de código remoto (RCE). El impacto requirió el despliegue de parches de emergencia globales en la cadena de software.

A07: Identification and Authentication Failures

- *Caso 1 (Disney+, 2019)*: A los pocos días del lanzamiento del servicio, miles de cuentas fueron secuestradas mediante ataques automatizados de "Credential Stuffing". La plataforma carecía en su lanzamiento de bloqueos dinámicos ante intentos masivos por IP y no obligaba al uso de MFA.

- *Caso 2 (Basecamp, 2022)*: Un bypass en el sistema de gestión de sesiones permitía a los atacantes mantener el acceso continuo a una cuenta web incluso si el usuario original cambiaba sus credenciales de seguridad principales, debido a la falta de revocación de tokens viejos en el servidor.

A08: Software and Data Integrity Failures

- *Caso 1 (SolarWinds, 2020)*: Hackers estatales inyectaron código malicioso firmado con certificados legítimos de SolarWinds en su software de monitoreo "Orion", provocando que más de 18,000 clientes descargaran una actualización comprometida creyendo que provenía de una fuente de integridad intacta.
- *Caso 2 (Malware en el Registro npm, 2021)*: Atacantes comprometieron paquetes populares de JavaScript de uso masivo (como ua-parser-js) modificando su código interno para desplegar mineros de criptomonedas y troyanos de robo de contraseñas de manera automatizada a los desarrolladores que descargaban la biblioteca sin validar su hash de integridad.

A09: Security Logging and Monitoring Failures

- *Caso 1 (Brecha de Datos de Marriott International, 2018)*: Atacantes persistieron en la red de la cadena Starwood durante 4 años antes de ser detectados de manera formal, permitiendo el robo progresivo de registros de 500 millones de huéspedes debido al nulo monitoreo de accesos extraños internos.
- *Caso 2 (Brecha de la OPM de EE. UU., 2015)*: La Oficina de Administración de Personal estadounidense sufrió la exfiltración de registros de millones de empleados gubernamentales. Los atacantes operaron sin ser detectados por más de un año debido a la carencia de herramientas centralizadas para auditar logs y detectar patrones de comportamiento anómalos.

A10: Server-Side Request Forgery (SSRF)

- *Caso 1 (Capital One, 2019)*: Una atacante explotó una vulnerabilidad de SSRF hospedada en un Web Application Firewall (WAF) mal configurado. Consiguió obligar al servidor de la aplicación a consultar el servicio de metadatos de AWS (<http://169.254.169.254/>), obteniendo las llaves maestras de acceso a la nube.
- *Caso 2 (Microsoft Exchange, 2021 - ProxyOracle)*: Una cadena de exploits crítica incluía un SSRF que permitía a atacantes externos enviar peticiones HTTP dirigidas al puerto interno del servidor de correo, logrando autenticarse como la cuenta de la máquina del sistema para tomar el control del dominio corporativo.

3. Código Vulnerable vs. Seguro (JavaScript)

Ejemplos de código para mostrar las 3 vulnerabilidades críticas (JavaScript):

Ejemplo 1: A03 - Inyección SQL (SQL Injection)

Código Vulnerable (Concatenación Directa)


```
// El input del usuario se concatena directamente en el query string

const express = require('express');

const app = express();

const db = require('./databaseConnection');

app.post('/login', (req, res) => {

    let username = req.body.username;

    let password = req.body.password;

    // MALO: Si username es: ' OR '1'='1 ', se altera la lógica del
    query

    let query = `SELECT * FROM usuarios WHERE user = '${username}' AND
    pass = '${password}'`;

    db.query(query, (err, result) => {

        if (result.length > 0) res.send("Bienvenido");

        else res.send("Credenciales incorrectas");

    });

});
```

Código Seguro (Consultas Parametrizadas / Prepared Statements)

```
// Se separan de manera segura los datos del usuario de las
instrucciones lógicas

app.post('/login', (req, res) => {

    let username = req.body.username;

    let password = req.body.password;
```

```
// BUENO: Se usan marcadores de posición (?) para sanitizar las variables

let query = 'SELECT * FROM usuarios WHERE user = ? AND pass = ?';

db.query(query, [username, password], (err, result) => {

  if (result.length > 0) res.send("Bienvenido");

  else res.send("Credenciales incorrectas");

});

});
```

Ejemplo 2: A01 - Broken Access Control (Insecure Direct Object Reference - IDOR)

Código Vulnerable (Falta de Verificación de Propietario)

```
// El sistema asume que el ID enviado en la URL le pertenece al usuario logueado

app.get('/api/invoice/:id', (req, res) => {

  let invoiceId = req.params.id;

  // MALO: Cualquiera puede cambiar el :id en la URL y leer facturas de otros usuarios

  db.query('SELECT * FROM facturas WHERE id = ?', [invoiceId], (err, result) => {

    res.json(result);

  });

});
```

Código Seguro (Validación del Contexto del Usuario Autenticado)

```
// Se valida explícitamente que el ID solicitado pertenezca al usuario en sesión

app.get('/api/invoice/:id', verifyToken, (req, res) => {

  let invoiceId = req.params.id;

  let userId = req.user.id; // Obtenido de forma segura a través del token JWT en la sesión

  // BUENO: El query restringe la búsqueda al propietario de la factura

  db.query('SELECT * FROM facturas WHERE id = ? AND usuario_id = ?', [invoiceId, userId], (err, result) => {

    if (result.length === 0) {

      return res.status(403).send("Acceso no autorizado a este recurso.");

    }

    res.json(result);

  });

});
```

Ejemplo 3: A05 - Security Misconfiguration (Exposición Extensa de Errores)

Código Vulnerable (Debug Mode y Stack Traces Expuestos en Producción)

```
// La aplicación maneja el error imprimiendo toda la estructura interna del sistema

app.get('/getUserData', (req, res) => {

  db.query('SELECT * FROM users WHERE id = ?', [req.query.id], (err, result) => {
```

```

        if (err) {

            // MALO: Revela rutas del backend, versiones del manejador
            de BD y consultas lógicas

            return res.status(500).json({ success: false, error:
err.stack });

        }

        res.json(result);

    });

});

```

Código Seguro (Mensajes Genéricos al Cliente y Logging Privado)

```

app.get('/getUserData', (req, res) => {

    db.query('SELECT * FROM users WHERE id = ?', [req.query.id], (err,
result) => {

        if (err) {

            // BUENO: Se guarda de forma segura en un log privado local
(para el programador)

            console.error(`Error interno de BD: ${err.stack}`);

            // Se le retorna al cliente un mensaje genérico sin dar
pistas del sistema interno

            return res.status(500).json({ success: false, message:
"Ocurrió un error interno en el servidor." });

        }

        res.json(result);

    });

});

```

4. Ranking Personal de las 5 Vulnerabilidades Más Críticas

A modo de criterio personal, este es el análisis estructurado de prioridades críticas en el ecosistema actual de software:

1. **A01: Broken Access Control (Control de Acceso Deficiente)**

La coloco como primer lugar, ya que es la vulnerabilidad más crítica ya que anula directamente la lógica de negocio básica de cualquier sistema. Ninguna criptografía o firewalls sirve si un atacante puede saltarse la capa de autorización con solo alterar manualmente un identificador numérico o un rol en la petición web, exponiendo directamente datos masivos a internet de manera trivial.

2. **A03: Injection (Inyección - SQL/Comandos)**

Aunque es una de las fallas más antiguas de la web, sigue estando presente debido a las malas prácticas de concatenación manual de cadenas. Su gravedad radica en que permite a un atacante no autenticado tomar el control administrativo absoluto del servidor de base de datos o ejecutar comandos en el sistema operativo subyacente.

3. **A06: Vulnerable and Outdated Components (Componentes Vulnerables y Obsoletos)**

En el desarrollo moderno basado en microservicios y Node.js, las aplicaciones dependen de cientos de librerías externas de código abierto. Si una de esas piezas secundarias tiene una falla pública, tu aplicación se vuelve inmediatamente vulnerable, transformando la cadena de suministro de software en un vector masivo de ataques silenciosos.

4. **A02: Cryptographic Failures (Fallas Criptográficas)**

El software maneja hoy en día grandes volúmenes de datos financieros y personales regulados por ley. Fallar en la criptografía adecuada implica que cualquier intrusión, robo de respaldos de base de datos o ataque de interceptación de red resulte en la exposición inmediata de secretos en texto claro, destruyendo la reputación de la empresa.

5. **A09: Security Logging and Monitoring Failures (Fallas de Registro y Monitoreo)**

Esta vulnerabilidad amplifica enormemente el impacto de todas las demás. Si los atacantes logran entrar al sistema y la aplicación no registra registros o alertas de este comportamiento anómalo, el atacante puede pasar desapercibido por meses o años robando información de manera persistente y borrando sus rastros de forma impune.

5. Conclusión

Comprender el OWASP Top 10 no debe interpretarse como una tarea exclusiva de los especialistas en ciberseguridad o auditores de infraestructura, sino como una competencia técnica indispensable para cualquier desarrollador de software en la actualidad. Tradicionalmente, la industria ha priorizado la velocidad de entrega de funcionalidades y la estética de la interfaz de usuario por encima de la robustez del código. Sin embargo, el panorama actual de ciberamenazas demuestra que un solo descuido en las etapas de codificación puede provocar catástrofes financieras, legales y de reputación para organizaciones globales. El OWASP Top 10 funciona como una guía estandarizada y un consenso global que nos permite identificar de raíz cuáles son los puntos débiles más explotados en el software web, permitiendo cambiar el paradigma hacia un enfoque preventivo en lugar de uno puramente reactivo.

En la práctica profesional del área de programación que podría tener en mi vida laboral, la aplicación de este conocimiento se traducirá en la adopción del enfoque "Shift-Left" (seguridad desde el inicio del diseño), integrando pruebas de seguridad estáticas y dinámicas dentro de nuestros pipelines de integración y despliegue continuo (CI/CD). Al programar el backend, la desinfección estricta de entradas y el uso obligatorio de consultas parametrizadas se volverán estándares de codificación por defecto, eliminando la inyección desde la raíz de la sintaxis. De igual forma, implementaré la verificación automatizada de dependencias obsoletas en los repositorios de código para blindar la cadena de suministro de software. Entender los riesgos de seguridad me capacita para diseñar arquitecturas de software robustas y resilientes que protejan de forma nativa la confidencialidad, disponibilidad e integridad de los datos de los usuarios.

Bibliografías

OWASP Top 10 (Documentación Base): OWASP Foundation. (2021). *OWASP Top 10:2021 - The Most Critical Web Application Security Risks*. <https://owasp.org/Top10/>

Casos Reales e Historial de Vulnerabilidades (Bases de Datos de Ciberseguridad):

- CVE Details. (2026). *The Ultimate CVE Cyber Security Vulnerability Database*. <https://www.cvedetails.com/>
- National Institute of Standards and Technology (NIST). (2026). *National Vulnerability Database (NVD)*. U.S. Department of Commerce. <https://nvd.nist.gov/>

Reportes Técnicos de Brechas de Seguridad Específicas:

- Amazon Web Services. (2019). *S3 Security Best Practices: Preventing Misconfigurations*. AWS Documentation.
- Apache Software Foundation. (2017). *Apache Struts 2 Documentation: Security Bulletin S2-045 (CVE-2017-5638)*. <https://cwiki.apache.org/confluence/display/WW/S2-045>
- CISA (Cybersecurity and Infrastructure Security Agency). (2021). *APT Actors Exploiting CVE-2021-44228 (Log4Shell)*. U.S. Department of Homeland Security.
- Federal Trade Commission (FTC). (2019). *The Equifax Data Breach Settlement*. U.S. Federal Government.
- U.S. House of Representatives Committee on Oversight and Government Reform. (2017). *The OPM Data Breach: How the Government Jeopardized Our National Security*.